

Project Report

Student Details:

Full Name - Vivek Kumar Mali

Roll Number - 24f2002719

Student Email : 24f2002719@ds.study.iitm.ac.in

Course - MAD-2 Project

About : Greeting everyone I am Vivek Kumar Mali, from Sirohi, Rajasthan, India. I am 20 years old and pursuing a BS degree in Data Science and Programming from IIT Madras , standalone. I am a passionate developer who enjoys solving real-world problems using Python and web technologies. I am especially interested in backend development and designing clean architectures.

Project Details:

Project Title : Hospital Management System

Problem Statement: Build a unified, web-based platform that automates workflows for Admins, Doctors, and Patients, ensuring efficient scheduling, secure data management, and streamlined communication.

Approach: The project employs a robust **Decoupled Client-Server Architecture**, connecting a headless Flask backend to a Vue.js frontend via secure RESTful APIs. It ensures strict data integrity through **Role-Based Access Control (RBAC)** while maintaining high system performance and scalability by offloading resource-intensive tasks to **Celery background workers** and optimizing high-traffic endpoints with **Redis caching**.

AI/LLM Declaration:

- I used 4% AI LLM Model (ChatGPT) in Bootstrap/CSS for Styling and layout
- I used 5% AI to resolve errors.
- I used 5% of backend in Faker module

Technologies and Framework Used:

Technology / Library	Purpose
Vue.js 3	Frontend framework for building the reactive Single Page Application (SPA)
Pinia	State management library for handling user sessions and global app state
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize trends such as activity vs. time
Flask	Core backend web framework for serving the REST API
Flask-RESTful	Handles authentication, password hashing, and Role-Based Access Control
SQLAlchemy	Object Relational Mapper (ORM) for interacting with the SQLite database
Celery	Distributed task queue for executing asynchronous background jobs

Redis	In-memory data store used as a Message Broker and Caching layer
SQLite	Lightweight local database for storing user data
MailHog	Email testing tool to intercept and display SMTP traffic locally
WeasyPrint	Rendering engine for generating PDF reports from HTML templates

DB Schema Design:

Tables:

1. **User:** Stores login credentials (**email**, **password**, **roles**).
2. **Doctor:** Linked to User; stores **specialization**, **experience**, and **bio**.
3. **Patient:** Linked to User; stores contact info and appointment history.
4. **Appointment:** The central entity linking **Doctor** and **Patient**. Attributes: **date**, **time**, **status** (Booked/Completed), **is_paid**.
5. **Treatment:** Linked to Appointment; stores medical records (**diagnosis**, **prescription**).
6. **DoctorAvailability:** Manages specific time slots per doctor to prevent double-booking.

Relationships:

- **Many-to-Many** : User → Role
- **One-to-One** : User → Doctor, User → Patient, Appointment → Treatment
- **Many-to-One** : Doctor → Specialization, Doctor → DoctorAvailability
- **One-to-Many** : Doctor → Appointment, Patient → Appointment

API Resource Endpoints:

Auth: **POST** /auth/login, **POST** /auth/register (Flask Token generation).

Users: **GET** /api/users (Admin), **POST** /api/doctors (Create Profile).

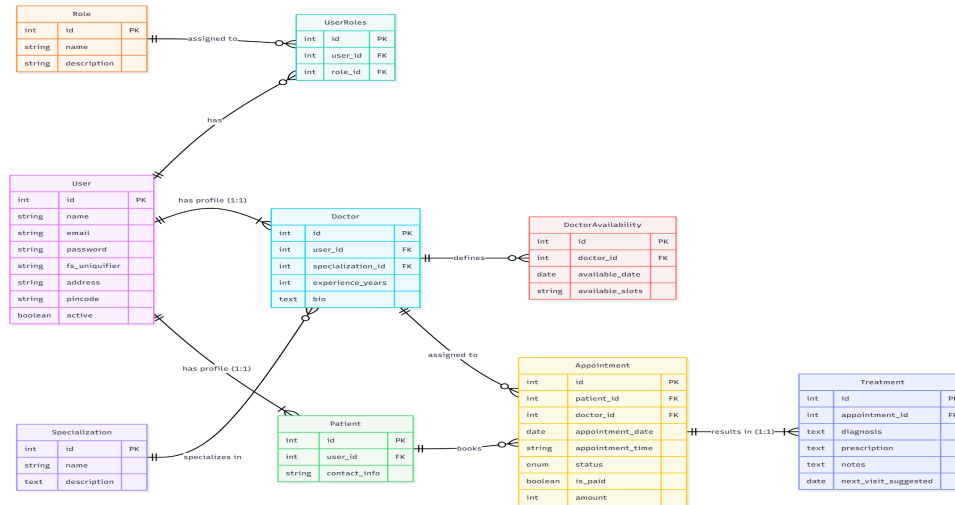
Appointments:

- **GET** /api/appointments: Fetches user-specific schedule.
- **POST** /api/appointments: Books a slot (Validates availability first).
- **GET** /api/doctors/<id>/slots: Checks real-time availability (Cached via Redis).

Operations:

- **POST** /api/treatments: Doctor adds diagnosis.
- **POST** /api/export/history: Triggers async CSV export job.
- **GET** /api/admin/analytics: Fetches data for dashboard charts

ER Diagram :



Architecture and Features:

A. Core Modules

1. **Admin Dashboard:**
 - **Analytics:** Visual charts (Chart.js) showing appointment trends and department demand.
 - **User Management:** Add doctors, manage profiles, and blacklist users.
2. **Doctor Portal:**
 - **Availability Manager:** Interactive UI to set weekly working hours.
 - **Consultation:** View assigned patients, complete appointments, and write digital prescriptions.
3. **Patient Portal:**
 - **Smart Booking:** Search doctors by specialization. The system prevents double-booking by validating slots in real-time.
 - **History & Payments:** View past treatments and make dummy payments for completed visits.

B. Backend Jobs (Celery)

1. **Daily Reminders:** Automatically emails patients at 8:00 AM if they have an appointment scheduled for the day.
2. **Monthly Reports:** Generates an HTML summary of total appointments and emails it to every doctor on the 1st of the month.
3. **Data Export:** Allows patients to trigger an asynchronous download of their medical history (CSV format) via email.

C. Performance & Optimization

- **Caching:** The "Public Doctor List" (10-min expiry) and "Available Slots" (1-min expiry) are cached in Redis to handle high traffic and reduce SQL queries.

Video Link: 📺 **MAD2 PROJECT HOSPITAL MANAGEMENT SYSTEM.mp4**

